ONLINE VOTING SYSTEM

Final-Year Project Report

Technology: HTML, CSS, JavaScript

Project category: AI/ML concept demonstration

Difficulty: Beginner / Easy

Prepared as an educational prototype. This report and application are not intended for real elections.

1. Abstract

The Online Voting System is a browser-based educational prototype intended to demonstrate a simple digital voting workflow. A user enters a demo voter ID, selects one of three sample candidates, and submits a vote. The application checks whether that demo ID has already voted, updates candidate totals, and displays percentages. A small AI-inspired module demonstrates explainable anomaly alerts by detecting repeated submissions using transparent rules. The system is implemented with HTML for structure, CSS for presentation, and JavaScript for validation and browser-local data handling. It is suitable for demonstrating front-end concepts, but it is not a secure or production-ready voting platform.

2. Problem Statement

Manual voting workflows can require time-consuming counting and result preparation. This project demonstrates how a basic web interface can record sample choices and automatically calculate totals. It also introduces the idea of flagging suspicious or repeated activity. The scope is limited to a local classroom demonstration and does not attempt to solve the identity, security, accessibility, or legal requirements of real elections.

3. Objectives

- Create a clear and responsive voting interface.
- Allow a user to select a sample candidate and submit a demo vote.
- Block repeat votes for the same demo ID in the current browser data.
- Calculate and display candidate totals and percentages.
- Demonstrate explainable, rule-based anomaly alerts.
- Document limitations and possible future improvements.

4. Scope

The prototype supports three sample candidates and stores demo votes in the browser's localStorage. Data remains local to the same browser profile. It has no server, verified voter registry, central database, administrator account, or secure ballot infrastructure. Resetting browser data can remove the records. These limitations are deliberate for an easy beginner project and mean it must never be used for real elections.

5. Technologies Used

HTML5 structures the page and form. CSS3 provides layout, responsive styling, candidate cards, progress bars, and status messages. JavaScript validates demo IDs, prevents duplicate IDs within the stored local data, calculates results, and renders alerts. Browser localStorage provides temporary persistence without a backend. No external JavaScript libraries are required.

6. Main Features

1. Candidate selection form.
2. Demo voter ID validation.
3. One-vote-per-ID check within the browser's local data.
4. Automatic vote totals and percentage bars.
5. Rule-based duplicate-attempt alerts.
6. Reset control for classroom testing.
7. Responsive design for desktop and mobile screens.

7. System Design and Workflow

Workflow: Open the page → enter a demo ID → select a candidate → validate the ID → check whether the ID already exists → if new, save the sample vote → calculate totals → refresh the results and alert panel. If the ID already exists, the application blocks the second submission and records a demo alert. All steps run in the browser.

8. AI and Machine Learning Component

The current project uses an explainable rule-based simulation rather than a trained ML model. Its duplicate-ID check illustrates anomaly detection at a conceptual level. It should not be described as a trained AI system. A future ML extension could use synthetic or properly authorized, privacy-protected data to train a model to identify unusual activity, then evaluate precision, recall, false-positive rate, and fairness. Any such model should only flag activity for human review, not silently invalidate legitimate votes. Sensitive voter information should not be used without a lawful basis and strong safeguards.

9. Setup and Execution

1. Extract the ZIP archive. 2. Open the folder containing index.html, style.css, and script.js. 3. Open index.html in a modern browser. 4. Enter a sample ID such as DEMO101, choose a candidate, and submit. 5. Submit DEMO101 again to see the duplicate alert. 6. Use Reset demo data to clear the browser's stored sample data. No server or package installation is required.

10. Testing

Test 1: Submit a valid ID and candidate — the vote should appear in totals. Test 2: Submit the same ID again — the duplicate attempt should be blocked and an alert shown. Test 3: Submit a different valid ID — the vote count should increase. Test 4: Enter an invalid or too-short ID — validation should show an error. Test 5: Reset data — counts and alerts should return to zero. Test 6: Resize the browser — the layout should adapt to a narrow screen.

11. Limitations and Security

The application is only a front-end demo. localStorage is user-controlled and can be modified or erased; client-side checks can be bypassed; voter identity is not verified; there is no server-side authorization, encrypted ballot handling, tamper-evident audit log, accessibility certification, or independent security review. Real voting systems require expert security, legal, operational, and accessibility review. Never enter real personal information into this prototype.

12. Future Enhancements

- Add a backend API and database for a controlled classroom simulation.
- Add authentication for a non-election demo, with secure session handling.
- Add server-side validation and audit logging.
- Build automated tests and accessibility checks.
- Add a genuine ML experiment using synthetic data and documented evaluation metrics.
- Conduct threat modeling and independent security testing before any real-world use.

13. Conclusion

The Online Voting System demonstrates a simple digital voting workflow using HTML, CSS, and JavaScript. It records sample votes, blocks repeat demo IDs within local browser data, and calculates visible results. The anomaly panel introduces explainable rule-based detection, while the report distinguishes that simulation from genuine machine learning. The project is appropriate for learning and demonstration, not for real voting.

14. References

- MDN Web Docs — HTML, CSS, JavaScript, and Web Storage API documentation: https://developer.mozilla.org/
- OWASP — general web application security guidance: https://owasp.org/